Lockstep Language — Sprint Retrospective

Project: Lockstep Language

Developer: Jey Lofton

Sprint Goal: Deliver a responsive front-end prototype demonstrating the Tell → Show → Do method and a simulated registration flow.

Completed: 11 / 11 story points (US-01, US-02, US-03)

---

1. What went well during the Sprint?

Starting with the Tell → Show → Do method as the backbone made a lot of other decisions easier. Once I knew every page had to serve one of those three stages, the information architecture almost wrote itself: Home for Tell, How It Works for Show, and Learning Method Details for Do. The three personas I built earlier kept me from drifting into features that sounded nice but did not help the visitor decide whether to register, which was the actual goal of the sprint.

On the implementation side, keeping the stack minimal — plain HTML, CSS, and vanilla JavaScript, no framework or build step — turned out to be the right call for the time I had. Client-side routing through goToPage() and localStorage gave the site a multi-page feel without the overhead of a bundler, and I could iterate quickly. The pronunciation and syllable-highlight demo was the single most satisfying piece to finish, because it is the clearest in-page example of what the method actually feels like.

Responsive work also landed earlier than I expected. Committing to Flexbox and two breakpoints up front (860px and 560px) kept the CSS small enough that I could reason about it the whole way through, and the hamburger menu behaved the way I wanted it to — closing on outside click, Escape, and window resize — without any special-case fixes.

2. What challenges did you encounter?

The honest answer is scope decisions, more than technical blockers. US-02 (registration) looks like one story on paper, but a real version of it involves a backend, authentication, email, and persistence — none of which fit this sprint. Deciding to deliver it as an explicitly simulated flow, and to label that clearly on the Register page and in the README rather than hide it, took longer to think through than the form itself did to build.

The pronunciation demo was the other place I spent extra time. Keeping the syllable highlights in sync with the audio was straightforward when the audio loaded; the harder part was deciding what should happen when it did not. I ended up adding a timed fallback animation so the demo still communicates the idea even if the MP3 fails, but settling on that behavior took more passes than I expected.

Working solo meant I had to be both the person writing the code and the person reviewing it. There is no second pair of eyes catching the small things, so I leaned on manual testing across viewport sizes and on re-reading my own acceptance criteria to decide when a story was really done.

3. What would you improve in the next Sprint?

I want to write things down earlier. For this sprint I kept a lot in my head — which was fine for three stories, but it would not scale. Even a lightweight sprint board kept up to date as I worked would have made it easier to see what

was actually "In Progress" versus "Done," and would have forced me to be more honest about partially-finished work (like the lesson-duration selector, which is a visual example rather than a wired-up control).

I would also like to introduce at least some automated testing next sprint, even if it is only a handful of unit tests around the validation logic in scripts/script.js. Manual testing worked for this scope, but it is the first thing that will stop scaling once registration becomes real.

Finally, I want to split larger stories earlier. US-02 at 5 points was really "build the form," "wire validation," "build the confirmation page," and "document that it is simulated." Breaking that into smaller tasks on the board would make my progress clearer and make the Done column feel earned step by step.

4. What did you learn about Scrum while developing your project?

The biggest thing I learned is that Scrum is less about ceremony and more about keeping the work visible and the goal narrow. Having a written Sprint Goal — "a responsive prototype that explains the method and lets a visitor register" — gave me something concrete to say no to things with. Features like adaptive lessons, pronunciation correction, and personalized progress tracking are on my roadmap, but they were not part of this sprint, and the Sprint Goal is what made that an easy call.

I also understand user stories better now than I did going in. Writing the acceptance criteria before building the pages forced me to define what "done" meant, and that is what let me honestly mark US-02 as Done while still being clear that it is a simulated flow. Done against the acceptance criteria is a different claim than "feature-complete for production," and Scrum gives you a vocabulary for keeping those two things separate.

Running Scrum as a one-person team also showed me which parts of the framework are really about communication inside a team (standups, planning meetings) and which parts carry their weight even when you are solo (the Sprint Goal, the board, the retrospective itself). The retrospective in particular is doing real work right now — it is making me articulate decisions I made intuitively, and that is the part I will carry into the next sprint.